

Table des aéronefs

Javions – étape 10

1. Introduction

Le but de cette étape est d’écrire le code permettant d’afficher la table des aéronefs, qui présente différentes informations à leur sujet.

2. Concepts

2.1. Table des aéronefs

La table des aéronefs, visible dans l’image ci-dessous, affiche un certain nombre d’informations au sujet des aéronefs. Les colonnes peuvent être réordonnées, et le contenu de la table peut être trié en fonction d’une ou plusieurs colonnes, en cliquant sur leur titre. Le bouton intitulé + tout à droite permet de choisir les colonnes affichées.

OACI	Indicatif	Immatriculation	Modèle	Type	Description	Longitude (°)	Latitude (°)	Altitude (m)	Vitesse (km/h)	+
34610D	VLG6ZA	EC-NCG	AIRBUS A-320neo	A20N	L2J	7,7538	46,6426	10 965	910	
3C64AA	DLH98H	D-AIEJ	AIRBUS A-321neo	A21N	L2J	3,6386	44,7897	10 371	904	
3950D1	AFR84PM	F-GUGR	AIRBUS A-318	A318	L2J	4,4029	45,1469	11 872	804	
3C6443	DLH27N	D-AIBC	AIRBUS A-319	A319	L2J	4,8244	45,3718	10 363	844	
344645	VLG11JA	EC-MEA	AIRBUS A-320	A320	L2J	6,0009	46,1658	1 791	447	
4402F2	EJU78CE	OE-IZE	AIRBUS A-320	A320	L2J	6,9115	46,5734	6 447	775	
3985A6	AFR96TP	F-HBNG	AIRBUS A-320	A320	L2J	5,0126	46,1580	10 371	790	
4CA15C	EIN4EK	EI-CVC	AIRBUS A-320	A320	L2J	8,4209	46,5226	10 858	720	
40123F	BAW724	G-EUUS	AIRBUS A-320	A320	L2J	6,1118	46,5178	3 025	469	
4D2417		9H-EWE	AIRBUS A-320	A320	L2J	6,2437	46,8934	11 582	878	
4B1A23	EZS262R	HB-JXJ	AIRBUS A-320	A320	L2J	7,1982	46,0461	8 405	810	
3C6424	CFG6MC	D-AIAD	AIRBUS A-321	A321	L2J	5,3534	46,0735	10 668	748	
471F62	WZZ1175	HA-LXA	AIRBUS A-321	A321	L2J	7,1783	46,1976	9 304	928	
3B7543		F-UJCT	AIRBUS A-330-200	A332	L2J	3,2819	46,3335	12 497	861	
4B1883	SWR78B	HB-JHJ	AIRBUS A-330-300	A333	L2J	6,3441	46,3982	1 806	342	
407739	TOM50T	G-TUMH	BOEING 737 MAX 8	B38M	L2J	6,2120	46,3080	9 007	310	

Figure 1 : Table des aéronefs (cliquer pour agrandir)

Par défaut, la table comporte les colonnes ci-dessous, dans cet ordre :

Titre	Contenu
OACI	Adresse OACI
Indicatif	Indicatif
Immatriculation	Immatriculation
Modèle	Modèle
Type	Indicateur de type
Description	Description
Longitude (°)	Longitude, en degrés, avec 4 décimales
Latitude (°)	Latitude, en degrés, avec 4 décimales

Altitude (m)	Altitude, en mètres, arrondie
Vitesse (km/h)	Vitesse, en kilomètres par heure, arrondie

Les six premières colonnes affichent des données textuelles, tandis que les quatre dernières affichent des données numériques.

À l'exception des colonnes contenant l'adresse OACI et la position (longitude et latitude) de l'aéronef, toutes les colonnes peuvent être vides dans le cas où l'information qu'elles devraient afficher est inconnue.

En effectuant un clic simple sur une ligne, il est possible de sélectionner l'aéronef auquel elle correspond. Dans la version finale du programme, cela aura pour effet de rendre son étiquette et sa trajectoire visible dans la vue des aéronefs.

En effectuant un clic double sur la table, il sera possible, dans la version finale du programme, de centrer la vue des aéronefs sur l'aéronef actuellement sélectionné – s'il y en a un.

3. Mise en œuvre Java

La mise en œuvre de cette étape est passablement plus simple que celle de la précédente. Néanmoins, les colonnes affichant des données textuelles sont moins difficiles à configurer que celles affichant des données numériques. Nous vous conseillons donc de faire une première version de votre table ne contenant que les six premières colonnes, la tester, puis d'ajouter les colonnes numériques dans un second temps.

3.1. Classe `AircraftTableController`

La classe `AircraftTableController` du sous-paquetage `gui`, publique et finale, gère la table des aéronefs. Son unique constructeur public prend en arguments :

- l'ensemble (observable mais non modifiable) des états des aéronefs qui doivent apparaître sur la vue – et qui provient bien entendu d'un gestionnaire d'états,
- une propriété JavaFX contenant l'état de l'aéronef sélectionné, dont le contenu peut être nul lorsqu'aucun aéronef n'est sélectionné – propriété qui est destinée à être identique à celle utilisée par la vue des aéronefs.

En plus de ce constructeur, `AircraftTableController` ne possède que deux méthodes publiques, qui sont :

- une méthode, nommée p. ex. `pane`, qui retourne le nœud à la racine de son graphe de scène – une instance de `TableView` comme expliqué plus bas,
- une méthode, nommée p. ex. `setOnDoubleClick`, qui prend en argument une valeur de type `Consumer<ObservableAircraftState>`, et qui appelle sa méthode `accept` lorsqu'un clic double est effectué sur la table et qu'un aéronef

est actuellement sélectionné, en lui passant en argument l'état de cet aéronef.

3.1.1. Graphe de scène

Le graphe de scène est extrêmement simple et consiste en une unique instance de `TableView` à laquelle la feuille de style `table.css` est attachée. Cette instance de `TableView` est configurée de manière à ce que :

- sa politique de redimensionnement des colonnes – que l'on peut changer au moyen de la méthode `setColumnResizePolicy` – soit égale à `CONSTRAINED_RESIZE_POLICY_SUBSEQUENT_COLUMNS`,
- le bouton permettant de sélectionner les colonnes à afficher soit visible, ce qui peut se faire au moyen de la méthode `setTableMenuButtonVisible`.

À chaque colonne de la table correspond une instance de `TableColumn`. Celles qui affichent des données numériques doivent avoir la classe de style `numeric` attachée, afin que leur contenu soit justifié à droite et pas à gauche.

Chaque colonne a une largeur préférée, qui lui est attribuée au moyen de sa méthode `setPrefWidth`. Pour les colonnes numériques, cette largeur vaut toujours 85, alors que pour les colonnes textuelles, elle varie et vaut : 60 (pour OACI), 70 (Indicatif), 90 (Immatriculation), 230 (Modèle), 50 (Type) et 70 (Description).

1. Colonnes textuelles

Les colonnes textuelles sont simples à créer car, en dehors de leur titre et de leur largeur préférée, le seul paramètre à configurer est leur « fabrique de valeur de cellule » (*cell value factory*) qui sert à obtenir, pour une ligne donnée, la valeur à afficher dans la cellule.

Cette configuration se fait en appelant la méthode `setCellValueFactory`, à laquelle on passe généralement une lambda. Cette lambda est appelée chaque fois qu'une nouvelle ligne est ajoutée à la table, afin de savoir ce que la colonne à laquelle cette lambda est associée doit afficher.

La lambda prend un seul argument, qui est du type `CellDataFeatures`. Il n'est pas nécessaire de comprendre ce type en détail, il faut juste savoir que sa méthode `getValue` permet d'obtenir la valeur correspondant à la ligne de la table en cours d'ajout, ici une valeur de type `ObservableAircraftState`.

Étant donné cet argument, qui correspond donc à une ligne entière, la lambda doit retourner une valeur observable de type `ObservableValue<String>`, qui est la chaîne de caractères à afficher dans la table.

Par exemple, pour la colonne affichant l'indicatif, la fabrique de valeur de cellule peut s'écrire ainsi :

```
TableColumn<ObservableAircraftState, String> c = ...;  
c.setCellValueFactory(f ->  
    f.getValue().callSignProperty().map(CallSign::string));
```

La méthode `getValue` de `f` retourne l'état observable de l'aéronef dont la ligne est en cours d'ajout – une valeur de type `ObservableAircraftState` – et on en extrait la propriété contenant l'indicatif au moyen de `callSignProperty`. Cela fait, il ne reste plus qu'à transformer le contenu de cette propriété – de type `CallSign` – en la chaîne à afficher dans la table, ce qui se fait au moyen de sa méthode `string`.

Pour comprendre pourquoi ce code fonctionne, il faut savoir que, comme expliqué dans [sa documentation](#), la méthode `map` n'appelle la lambda qu'on lui a passée que lorsque la valeur de la propriété à laquelle l'a appliquée n'est pas nulle. Sinon, la valeur observable qu'elle retourne produit simplement `null`. De plus, `TableColumn` n'affiche rien dans la cellule lorsque la valeur observable qui donne son contenu produit `null`.

Dès lors, tant que le contenu de `callSignProperty` est `null` – généralement car aucun message d'identification n'a été reçu de l'aéronef correspondant – la cellule de la table est vide. Mais lorsque le contenu de `callSignProperty` est différent de `null`, et est donc une instance de `CallSign`, la méthode `string` est appelée sur elle, et la chaîne retournée est affichée dans la cellule de la table.

2. Colonnes numériques

Les colonnes numériques sont plus difficiles à définir que les textuelles pour différentes raisons, détaillées ci-dessous.

Une première (petite) difficulté est que la classe de style `numeric` doit être attachée aux colonnes numériques, afin que leur contenu soit bien aligné à droite.

Une seconde difficulté est liée au fait que l'on désire contrôler le nombre de décimales affichées pour les différentes colonnes numériques, qui doit être 4 pour la longitude et la latitude, et 0 pour les autres valeurs. JavaFX n'offre aucun moyen de faire cela, et il nous faut donc trouver comment contourner cette limitation.

La solution que nous vous proposons d'utiliser consiste à représenter le contenu de ces cellules au moyen de chaînes de caractères, exactement comme les cellules textuelles. Cela nous permet de contrôler la manière dont les valeurs numériques sont transformées en chaînes, en particulier le nombre de décimales à utiliser.

Afin de transformer les valeurs numériques en chaînes, et inversement, nous utiliserons la classe `NumberFormat`. Une nouvelle instance de cette classe peut être obtenue au moyen de la méthode `getInstance`, et le nombre de décimales à utiliser configuré au moyen des méthodes `setMinimumFractionDigits` et

`setMaximumFractionDigits`. Une fois configurée, l'instance de `NumberFormat` peut être utilisée pour convertir des valeurs numériques en chaînes au moyen de `format`, et pour convertir des chaînes en valeur numérique au moyen de `parse` – ce qui est utile pour le comparateur décrit ci-après.

Le fait de convertir les valeurs numériques en chaînes nous permet de contrôler leur format mais pose un problème pour le tri des colonnes, puisque trier les représentations textuelles d'une séquence de nombres n'est pas équivalent à trier les nombres eux-mêmes. Il faut donc modifier le comparateur associé aux colonnes numériques, au moyen de la méthode `setComparator`. On lui passe un comparateur qui compare les chaînes qu'il reçoit comme des chaînes si au moins l'une d'entre elles est vide, et comme des nombres sinon.

3.1.2. Liens et auditeurs

Comme pour la vue des aéronefs, un auditeur doit être installé sur l'ensemble des états d'aéronefs passé au constructeur, afin de correctement faire apparaître et disparaître les aéronefs correspondants de la table. Cela implique d'appeler les méthode `add` et `remove` de la liste retournée par la méthode `getItems` de `TableView`. De plus, afin d'essayer de préserver autant que possible l'ordre de tri de la table, l'auditeur ajoutant un état à la table doit appeler sa méthode `sort` une fois l'ajout effectué.

Pour gérer la sélection d'un aéronef, il faut encore installer deux auditeurs :

- le premier sur la propriété passée au constructeur, qui appelle d'une part la méthode `select` du modèle de sélection retourné par la méthode `getSelectionModel` de `TableView` afin de sélectionner l'aéronef dans la table, et également la méthode `scrollTo` de `TableView` si l'état n'est pas déjà celui sélectionné – afin de le rendre visible,
- le second sur la propriété `selectedItemProperty` du modèle de sélection, afin de garantir que, lorsque l'utilisateur clique sur une ligne de la table, la propriété passée au constructeur soit modifiée pour contenir l'état correspondant.

3.1.3. Gestion des événements

Le seul événement à gérer dans la table est celui d'un clic double, avec le bouton gauche de la souris, sur la table. Un tel clic doit provoquer l'appel de la méthode `accept` du consommateur passé à `setOnDoubleClick` – si cette méthode a été appelée – avec l'état de l'aéronef sélectionné dans la table, s'il y en a un.

Pour installer le gestionnaire d'événement correspondant, on utilise la méthode `setOnMouseClicked` sur l'instance de `TableView`. Lorsqu'il est appelé, il faut utiliser les méthodes `getClickCount` et `getButton` de l'événement qui lui est passé afin de vérifier qu'il s'agit bien d'un clic double, et que le bouton pressé est bien le bouton gauche (`PRIMARY`).

3.1.4. Conseils de programmation

Les cellules de la table affichant l'adresse OACI d'un aéronef, de même que toutes celles affichant des données provenant de la base de données micronics, ont un contenu constant. Cela pose un petit problème, car les « fabriques de valeur de cellule » doivent retourner une valeur observable. Il faut donc trouver comment transformer une valeur quelconque en une valeur observable constante.

Il existe plusieurs techniques pour faire cela, la plus simple étant probablement de créer une instance de `ReadOnlyObjectWrapper` en lui passant en argument la valeur constante, comme illustré par cet extrait de programme :

```
ObservableValue<String> constantObservable =  
    new ReadOnlyObjectWrapper<>("Javions");
```

3.2. Tests

Pour tester cette étape, vous pouvez adapter le programme de test de l'étape précédente afin de remplacer l'empilement de la vue des aéronefs et de la carte de base par la table des aéronefs. En exécutant votre programme, vous devriez voir une fenêtre ressemblant à celle de la figure 1, et les valeurs affichées devraient se mettre à jour de manière similaire à celle du test de l'étape 7, mais beaucoup plus rapidement.

4. Résumé

Pour cette étape, vous devez :

- écrire la classe `AircraftTableController` selon les indications données ci-dessus,
- tester votre code,
- documenter la totalité des entités publiques que vous avez définies.

Aucun rendu n'est à faire pour cette étape avant le rendu final. N'oubliez pas de faire régulièrement des copies de sauvegarde de votre travail en suivant [nos indications à ce sujet](#).